VILNIUS GEDIMINAS TECHNICAL UNIVERSITY
FACULTY OF FUNDAMENTAL SCIENCES
DEPARTMENT OF INFORMATION TECHNOLOGIES

Shashikanth Hosur Ramegowda, Shaeed Khan, Rimsha Tariq

TRANSFORMERIAI IR JŲ TAIKYMAS NATŪRALIOSIOS KALBOS APDOROJIMO (NLP) UŽDUOTYSE

TRANSFORMERS AND THEIR APPLICATION IN NATURAL LANGUAGE PROCESSING (NLP) TASKS

Group Project Paper

Engineering of Artificial Intelligence study program, state code 6211BX023

Informatics study field

Vilnius, 2026

VILNIUS GEDIMINAS TECHNICAL UNIVERSITY
FACULTY OF FUNDAMENTAL SCIENCES
DEPARTMENT OF INFORMATION TECHNOLOGIES

Shashikanth Hosur Ramegowda, Shaeed Khan, Rimsha Tariq

TRANSFORMERIAI IR JŲ TAIKYMAS NATŪRALIOSIOS KALBOS APDOROJIMO (NLP) UŽDUOTYSE

TRANSFORMERS AND THEIR APPLICATION IN NATURAL LANGUAGE PROCESSING (NLP) TASKS

Group Project Paper

Engineering of Artificial Intelligence study program, state code 6211BX023

Informatics study field

Vilnius, 2026

Annotation

This project paper studies the Transformer architecture and its application to natural language processing (NLP) tasks. The theoretical part explains how self-attention, multi-head attention, positional encoding and large-scale pre-training replaced recurrent neural networks as the dominant approach in NLP, and reviews the main families of Transformer models (encoder-only BERT-type models, decoder-only GPT-type models and encoder–decoder models such as T5 and BART). The literature review covers international research and research carried out in Lithuania, including work on Lithuanian sentiment analysis, the trilingual LitLat BERT model and the first open Lithuanian large language models. It concludes that most published comparisons are done on GPU servers with large models, while evidence on what Transformers offer under realistic resource limits (a laptop CPU, a few thousand labelled examples) is scarce.

The practical part answers this question experimentally on binary sentiment classification of movie-review sentences (Stanford Sentiment Treebank, SST-2). Ten models from four families were implemented in Python (PyTorch, Hugging Face Transformers, scikit-learn) and compared under one protocol: three classical TF-IDF models, a bidirectional LSTM trained from scratch, three Transformer encoders that we fine-tuned ourselves on a CPU (BERT-tiny, BERT-small and DistilBERT), and three ready-made Transformer models used without any training (an in-domain DistilBERT, an out-of-domain Twitter RoBERTa and a zero-shot natural language inference model). Models were evaluated by accuracy, macro F1, training time, inference latency and size, as well as on linguistically difficult subsets with negation and contrast. A learning-curve experiment measured how much labelled data each approach needs. DistilBERT fine-tuned on only 8,000 sentences reached 88.8% accuracy and significantly outperformed the best classical model trained on all 65,349 sentences (82.9%), while the ready-made in-domain DistilBERT reached 91.1%. Compact Transformers only matched the classical models, and the recurrent network without pre-training gave no advantage. Pre-training made Transformers far more data-efficient (+14.1 percentage points with 500 labelled sentences) and more robust on negation, at the cost of 20–2,000 times slower inference on a CPU. Practical recommendations for choosing a model are given.

Keywords: Transformer, self-attention, BERT, DistilBERT, fine-tuning, transfer learning, sentiment analysis, natural language processing.

Contents

Introduction 3
1. Literature Review 5
1.1. Search strategy 5
1.2. From word vectors to the Transformer 5
1.3. Families of pre-trained Transformer models 5
1.4. Transformers for sentiment analysis and text classification 6
1.5. Research in Lithuania 6
1.6. Comparative summary 7
1.7. Conclusions of the literature review 7
2. Research Methodology 9
2.1. Problem formulation 9
2.2. Dataset 9
2.3. Compared models (competitors) 9
2.4. Evaluation criteria 10
2.5. Design and structure of the developed program 11
2.6. The Transformer architecture 12
2.7. Mathematical methods of training and evaluation 14
2.8. Experimental protocol and threats to validity 16
3. Practical Part: Implementation and Results 17
3.1. Implementation of the program 17
3.2. Overall comparison of the models 17
3.3. Cost: training time, inference latency and model size 19
3.4. Data efficiency of pre-training 19
3.5. Ready-made and zero-shot models 20
3.6. Error analysis: negation and contrast 20
3.7. Comparison with previous research 22
Conclusions 23
References 24
Appendices 26
Appendix A. Source code and reproduction of the results 26

List of Figures

Fig. 1. Structure of the developed program (pipeline of Python modules) 11
Fig. 2. Transformer encoder with a classification head, as used for sentiment classification 12
Fig. 3. Test accuracy of all compared models (colour shows the model family) 18
Fig. 4. Accuracy versus inference latency on CPU (bubble area proportional to the number of parameters) 19
Fig. 5. Learning curves: test accuracy versus number of labelled training sentences 20
Fig. 6. Accuracy on all sentences and on sentences with negation and contrast 21
Fig. 7. Self-attention weights of DistilBERT for an example sentence (last layer, mean over heads) 22

List of Tables

Table 1. Distribution of work among the team members 4
Table 2. Summary of the reviewed works most relevant to this project 7
Table 3. Data splits used in the experiment (SST-2) 9
Table 4. Compared models (competitors) 10
Table 5. Fine-tuning hyper-parameters 15
Table 6. Results on the SST-2 test set (872 sentences), sorted by accuracy 17
Table 7. Bootstrap 95% confidence intervals and McNemar test against the best model 18
Table 8. Test accuracy as a function of the number of labelled training examples 20
Table 9. Accuracy (%) on linguistically difficult subsets of the test set 21

Acronyms and Terms

AdamW	Adam optimiser with decoupled weight decay
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory network
CI	Confidence interval
CLS	Special classification token of BERT-type models
CPU / GPU	Central / Graphics processing unit
GPT	Generative Pre-trained Transformer
LLM	Large language model
MLM	Masked language modelling
NB	Naive Bayes
NLI	Natural language inference
NLP	Natural language processing
RNN	Recurrent neural network
SST-2	Stanford Sentiment Treebank, binary version
SVM	Support vector machine
TF-IDF	Term frequency – inverse document frequency

Introduction

Relevance of the research. Natural language processing (NLP) is the branch of artificial intelligence that enables computers to analyse, understand and generate human language. For decades the field relied on hand-crafted features and statistical models such as n-gram language models, naive Bayes classifiers and support vector machines. In the 2010s these were gradually replaced by neural networks: word embeddings such as word2vec (Mikolov et al., 2013) gave words dense vector representations, and recurrent neural networks, in particular long short-term memory networks (Hochreiter & Schmidhuber, 1997), became the standard way to model sentences. Recurrent networks, however, process text one token at a time, which makes training slow and makes it difficult to capture dependencies between words that are far apart.

In 2017 Vaswani et al. proposed the Transformer, an architecture that discards recurrence and relies only on the attention mechanism: every token can directly "look at" every other token, and all positions are processed in parallel. Within two years the Transformer became the foundation of almost every state-of-the-art NLP system. BERT (Devlin et al., 2019) showed that a Transformer encoder pre-trained on unlabelled text can be fine-tuned to beat previous results on eleven language understanding tasks, and the GPT series (Radford et al., 2018; Brown et al., 2020) showed that a Transformer decoder trained to predict the next word becomes a general-purpose text generator. Today Transformers power machine translation, search engines, chat assistants and document analysis systems.

The topic is relevant for three reasons. First, Transformers are now the default tool of NLP practitioners, so engineers need to understand both how they work and when they are worth their cost. Second, the computational price of these models is high (Strubell et al., 2019), so an informed choice between a small classical model, a compact Transformer and a large pre-trained model has a real economic and environmental impact. Third, for smaller languages such as Lithuanian, the question of how well Transformer methods work with limited labelled data is directly connected to the digital sustainability of the language (Ulčar & Robnik-Šikonja, 2021; Nakvosas et al., 2024).

Research problem. Published benchmarks usually report the accuracy of the largest models trained on GPU clusters with the full training set. A student, a small company or a public institution, by contrast, often has only an ordinary computer, a limited number of labelled examples and a need for fast predictions. The research problem is therefore: how large is the practical advantage of Transformer models over classical and recurrent approaches for a typical NLP task (sentiment classification), and at what cost in training time, inference latency and model size, when all experiments are carried out on a standard CPU-only laptop?

Research object. Transformer-based language models applied to text classification.

Research aim. To analyse the Transformer architecture and its applications in NLP and to evaluate experimentally the effectiveness of Transformer-based models for sentiment classification in comparison with classical machine learning and recurrent neural network methods.

Objectives to achieve the aim:

1. To review scientific literature on Transformer models and their applications in NLP, in Lithuania and worldwide, and to identify research gaps.
2. To describe the theoretical foundations of the Transformer architecture (self-attention, multi-head attention, positional encoding, pre-training and fine-tuning).
3. To define the experimental methodology: dataset, compared models (competitors), evaluation criteria and mathematical methods.
4. To implement classical, recurrent and Transformer-based classifiers in Python and run a comparative experiment on the SST-2 dataset under equal conditions.

5. To analyse the results in terms of quality, data efficiency, speed and model size, including an error analysis on sentences with negation and contrast, and to formulate practical recommendations.

Research methods. Analysis of scientific literature, comparative analysis, computational experiment (training and evaluation of models), statistical evaluation of the results (bootstrap confidence intervals, McNemar's test) and visualisation of attention weights.

Structure of the work. The paper consists of an introduction, three chapters, conclusions, a list of references and an appendix. Chapter 1 reviews the literature, Chapter 2 describes the research methodology (problem, dataset, competitors, criteria, program design and mathematical methods), and Chapter 3 presents the implementation and the results of the experiments. The source code is publicly available in the project repository (Appendix A).

Distribution of work. The project was carried out by a team of three students. All members took part in formulating the problem, discussing the results and editing the final text; the main responsibilities are shown in Table 1.

Table 1. Distribution of work among the team members

Team member	Main responsibilities
Shashikanth Hosur Ramegowda	Problem formulation and experiment design; implementation of the Transformer fine-tuning program; practical part (Chapter 3); presentation
Shaeed Khan	Literature search in Google Scholar and Web of Science; literature review (Chapter 1); classical and BiLSTM baselines; references
Rimsha Tariq	Introduction; theory of the Transformer architecture and mathematical methods (Chapter 2); evaluation of ready-made models; error analysis; conclusions

1. Literature Review

1.1. Search strategy

The literature was collected from Google Scholar and Web of Science, supplemented with the ACL Anthology (the main open archive of computational linguistics) and arXiv for the most recent preprints. The core search query was:

("transformer" OR "BERT" OR "self-attention" OR "pre-trained language model") AND ("natural language processing" OR "text classification" OR "sentiment analysis")

For the Lithuanian part the query was narrowed with the terms ("Lithuanian" OR "Baltic languages"). Both databases return thousands of records for the core query, so the results were sorted by relevance and citation count and screened by title and abstract. The inclusion criteria were: (1) the publication proposes or evaluates a Transformer-based model or an important predecessor; (2) it reports results on a text classification or language understanding task, or describes a widely used model; (3) it is peer-reviewed or a widely cited technical report. Seminal works (the original Transformer, BERT, GPT, RoBERTa, DistilBERT and others) were added through backward citation tracking. About 40 publications were kept for analysis; the most important of them are summarised in Table 2.

1.2. From word vectors to the Transformer

Early neural approaches represented each word by a static vector, learned with word2vec (Mikolov et al., 2013) or similar methods, and combined these vectors with convolutional or recurrent networks. Such vectors cannot distinguish the meanings of an ambiguous word in different contexts. Bahdanau et al. (2015) introduced the attention mechanism for machine translation, allowing the decoder of a recurrent model to focus on the relevant parts of the source sentence. Contextual representations followed: ELMo (Peters et al., 2018) computed word vectors with a deep bidirectional LSTM language model, and ULMFiT (Howard & Ruder, 2018) showed that a pre-trained language model can be fine-tuned for text classification with very little labelled data. These works established the "pre-train, then fine-tune" paradigm that the Transformer later made dominant.

Vaswani et al. (2017) proposed the Transformer, an encoder–decoder model built entirely from self-attention and feed-forward layers. On English–German and English–French translation it achieved better quality than recurrent and convolutional systems at a fraction of the training cost, because all positions of a sentence are processed in parallel. The architecture is described in detail in Chapter 2.

1.3. Families of pre-trained Transformer models

Three families of models grew out of the original architecture.

Encoder-only models. BERT (Devlin et al., 2019) pre-trains a Transformer encoder with masked language modelling, in which 15% of the tokens are hidden and the model has to predict them from the context on both sides. BERT-base (110 million parameters) and BERT-large (340 million) set new records on the GLUE benchmark (Wang et al., 2019), including SST-2 sentiment classification. RoBERTa (Liu et al., 2019) showed that BERT was under-trained and obtained better results simply by training longer on more data, with larger batches and without the next-sentence prediction task. ALBERT (Lan et al., 2020) reduced the number of parameters through factorised embeddings and cross-layer parameter sharing; ELECTRA (Clark et al., 2020) replaced masked language modelling with a more sample-efficient "replaced token detection" objective; DeBERTa (He et al., 2021) separated content and position information in attention and was the first model to exceed the human baseline on the SuperGLUE benchmark. XLM-RoBERTa (Conneau et al., 2020) extended the approach to 100 languages, including Lithuanian.

Compact encoders. Because large encoders are expensive at inference time, a line of research focuses on compression. DistilBERT (Sanh et al., 2019) uses knowledge distillation to train a 6-layer

student from the 12-layer BERT teacher; it is 40% smaller and 60% faster while retaining about 97% of BERT's language understanding performance. Turc et al. (2019) released a set of 24 compact BERT models (from BERT-tiny with 2 layers to BERT-base) and showed that pre-training compact models directly is important for good results. Tay et al. (2022) survey "efficient Transformers" that reduce the quadratic cost of attention for long sequences.

Decoder-only and encoder–decoder models. GPT (Radford et al., 2018) pre-trains a Transformer decoder as a left-to-right language model. GPT-3 (Brown et al., 2020), with 175 billion parameters, showed that a sufficiently large model can perform new tasks from instructions and a few examples placed in the prompt (in-context learning) without any fine-tuning. Instruction tuning with human feedback made such models usable as assistants. Encoder–decoder models such as T5 (Raffel et al., 2020), which casts every task as text-to-text, and BART (Lewis et al., 2020) are particularly strong for generation tasks such as summarisation and translation.

1.4. Transformers for sentiment analysis and text classification

Sentiment analysis is one of the most studied text classification tasks (Zhang et al., 2018). Its most widely used benchmark is the Stanford Sentiment Treebank (Socher et al., 2013). Classical methods based on bag-of-words or TF-IDF features with linear classifiers (Joachims, 1998) reach about 79–83% accuracy on SST-2, recurrent and convolutional networks reach 84–88%, and fine-tuned Transformers exceed 90%: BERT-base reports 93.5%, RoBERTa-large 96.4% and the largest T5 model 97.5% on the GLUE test set.

Sun et al. (2019) studied how to fine-tune BERT for text classification and showed that the choice of learning rate, layer-wise learning rates and further in-domain pre-training strongly influence the result. Yin et al. (2019) proposed zero-shot text classification by reformulating classification as natural language inference (NLI): the text is the premise and a sentence such as "This text is about sport" is the hypothesis. This allows a model trained on NLI data to classify text into arbitrary labels without any task-specific training examples. Loureiro et al. (2022) trained RoBERTa models on Twitter data and showed that domain-specific pre-training matters for social-media text. Zhang et al. (2024) evaluated large language models such as ChatGPT on sentiment tasks and found that, while zero-shot LLMs are competitive on simple binary sentiment, smaller fine-tuned models still outperform them on tasks that require structured or fine-grained sentiment understanding.

A separate stream of work analyses what Transformers learn. Clark et al. (2019) and Rogers et al. (2020) showed that individual attention heads in BERT specialise in syntactic relations such as direct objects or coreference, whereas Jain and Wallace (2019) warned that attention weights should not automatically be interpreted as explanations of a prediction.

1.5. Research in Lithuania

Lithuanian is a morphologically rich language with free word order and relatively few annotated resources, which makes it a challenging case for NLP. Kapočiūtė-Dzikienė et al. (2013) compared machine learning and lexicon-based approaches for sentiment classification of Lithuanian Internet comments and found that the specifics of the language (rich inflection, non-normative spelling in comments) strongly affect the choice of features. In a later study, Kapočiūtė-Dzikienė, Damaševičius and Woźniak (2019) compared traditional classifiers (multinomial naive Bayes, support vector machines) with deep neural networks (LSTM and CNN with word embeddings) on a dataset of Lithuanian comments in a three-class setting (positive, negative, neutral). Interestingly, the best traditional method (naive Bayes, about 0.735 accuracy) outperformed the best deep learning model (CNN with FastText embeddings, about 0.706), showing that neural models without pre-training do not automatically win when data are limited.

The arrival of Transformers changed the situation for Baltic languages. Ulčar and Robnik-Šikonja (2021) trained LitLat BERT, a trilingual Lithuanian–Latvian–English model based on the XLM-RoBERTa architecture, and showed that the size of the training corpus and of the vocabulary strongly determine the quality of BERT models for the Baltic languages; models trained for a few

languages outperform massively multilingual models on most tasks. Vileikytė, Lukoševičius and Stankevičius (2024) from Kaunas University of Technology fine-tuned multilingual BERT and T5 models for sentiment analysis of Lithuanian online reviews from several domains, reporting high accuracy on clearly polar reviews and lower accuracy on ambiguous middle ratings. Nakvosas, Daniušis and Mulevičius (2024) from the Vilnius company Neurotechnology released the first open Llama2-based large language models for Lithuanian (7 and 13 billion parameters), pre-trained on more than 14 billion Lithuanian tokens, together with translated evaluation benchmarks.

1.6. Comparative summary

Table 2 summarises the most relevant works. It is later used to place the results of our experiments in context.

Table 2. Summary of the reviewed works most relevant to this project

Authors (year)	Model / approach	Task, data	Main result / relevance
Socher et al. (2013)	Recursive neural tensor network	SST sentiment, EN	Created SST; NB ≈ 82%, SVM ≈ 79%, RNTN 85.4% (binary)
Vaswani et al. (2017)	Transformer (self-attention only)	Machine translation	Better quality than RNN/CNN at lower training cost
Devlin et al. (2019)	BERT, masked LM pre-training	GLUE incl. SST-2	SST-2 test 93.5% (base) / 94.9% (large)
Liu et al. (2019)	RoBERTa	GLUE	Longer training, more data: SST-2 96.4%
Sanh et al. (2019)	DistilBERT (distillation)	GLUE	40% smaller, 60% faster, ~97% of BERT quality
Turc et al. (2019)	Compact BERTs (tiny…base)	GLUE, distillation	Pre-training compact models matters
Yin et al. (2019)	Zero-shot via NLI entailment	Topic, emotion, situation	Classification without task-labelled data
Brown et al. (2020)	GPT-3, in-context learning	Many NLP tasks	Few-shot learning without fine-tuning
Kapočiūtė-Dzikienė et al. (2019)	NB, SVM vs LSTM, CNN	Lithuanian comments, 3 classes	Traditional NB 0.735 > best deep 0.706 accuracy
Ulčar & Robnik-Šikonja (2021)	LitLat BERT (XLM-R based)	LT, LV, EN tasks	Trilingual model beats massively multilingual
Vileikytė et al. (2024)	Fine-tuned BERT and T5	Lithuanian reviews, multi-domain	High accuracy on clearly polar reviews
Nakvosas et al. (2024)	Open Llama2 7B/13B for Lithuanian	LT language modelling	First open Lithuanian LLMs, 14B+ tokens
Zhang et al. (2024)	LLMs (ChatGPT etc.) vs fine-tuned	13 sentiment tasks	Small fine-tuned models still win on complex tasks

1.7. Conclusions of the literature review

The review leads to the following conclusions, which define the gap addressed by this project:

– Transformer models are the state of the art for text classification; on SST-2 fine-tuned Transformers exceed 90% accuracy, compared with roughly 79–83% for classical models.

– Almost all reported results are obtained on GPU hardware with the full training set. The cost side of the comparison (training time on ordinary hardware, inference latency, model size) is rarely reported together with accuracy.

– Lithuanian studies show that without pre-training, deep networks do not necessarily outperform classical methods on small datasets (Kapočiūtė-Dzikienė et al., 2019), while recent Transformer-based work (Ulčar & Robnik-Šikonja, 2021; Vileikytė et al., 2024) indicates that pre-training is the key factor. The data efficiency of compact pre-trained models under a fixed budget of labelled examples is insufficiently studied.

– Ready-made models available in public model hubs (off-the-shelf, cross-domain and zero-shot) are increasingly used in practice instead of training, but their behaviour compared with a model fine-tuned on in-domain data is seldom measured under one protocol.

Therefore, the practical part of this project compares classical, recurrent, fine-tuned Transformer and ready-made Transformer models under identical conditions on a CPU-only computer, measuring both quality and cost, and studies how the advantage of pre-training depends on the amount of labelled data.

2. Research Methodology

2.1. Problem formulation

The practical task is binary sentiment classification of English sentences. Given a sentence $x = (w_1, w_2, \ldots, w_n)$, the goal is to learn a function $f(x) \rightarrow y$, where $y \in \{0, 1\}$ denotes negative or positive sentiment. The classifier is learned from a labelled training set $D = \{(x_i, y_i)\}$, $i = 1, \ldots, N$, and its quality is measured on a separate test set that is never used for training or model selection.

Sentiment classification was chosen for three reasons. First, it is one of the most common real-world applications of NLP (customer reviews, social media monitoring, product feedback). Second, it is a standard benchmark with published results for all model families, so our results can be compared with the literature (Table 2). Third, the task contains linguistic phenomena that are difficult for simple models, such as negation ("not good"), contrast ("the plot is weak, but the acting is great") and sarcasm, which makes it a good test of the contextual understanding provided by self-attention.

The experiment is designed to answer the following research questions:

– RQ1. How much more accurate are Transformer models than classical and recurrent models on the same data?

– RQ2. What is the cost of this improvement in training time, inference latency and model size on CPU-only hardware?

– RQ3. How does the advantage of a pre-trained Transformer depend on the number of labelled training examples?

– RQ4. How well do ready-made models (off-the-shelf in-domain, out-of-domain and zero-shot) perform compared with models trained on the task data?

– RQ5. Are Transformers better at linguistically difficult sentences with negation and contrast?

2.2. Dataset

The Stanford Sentiment Treebank (SST) was built by Socher et al. (2013) from 11,855 sentences of movie reviews from the website Rotten Tomatoes. Each sentence was parsed into a tree and every phrase was annotated by three human judges. The binary version SST-2, included in the GLUE benchmark (Wang et al., 2019), removes neutral sentences and contains 67,349 training phrases and sentences and 872 validation sentences. Test labels of GLUE are hidden, therefore, following common practice, we used the official validation set as our held-out test set. For model selection (choosing the best epoch) we randomly separated 2,000 examples from the training data as a development set, stratified by class. The resulting splits are shown in Table 3.

Table 3. Data splits used in the experiment (SST-2)

Split	Source	Size	Positive share	Mean length, words
Train	SST-2 train minus dev	65 349	55.8%	9.4
Dev	2 000 random from SST-2 train	2 000	55.8%	9.6
Test	SST-2 official validation	872	50.9%	19.5

The training data are slightly imbalanced towards the positive class (about 56%), while the test set is almost balanced. The training set contains many short phrases (the average length is below 10 words), whereas the test set contains complete sentences of about 19 words on average. This difference between training and test distributions is part of the challenge: models must generalise from phrases to whole sentences.

2.3. Compared models (competitors)

Ten models were selected to represent four approaches that a practitioner can realistically choose from. They are listed in Table 4.

Table 4. Compared models (competitors)

Family	Model	Pre-trained	Trained by us	Parameters
Classical	TF-IDF + Multinomial Naive Bayes	no	yes (65k)	0.1 M*
Classical	TF-IDF + Logistic Regression	no	yes (65k)	74 k*
Classical	TF-IDF + Linear SVM	no	yes (65k)	74 k*
Recurrent	BiLSTM, random embeddings	no	yes (65k)	2.0 M
Transformer, fine-tuned	BERT-tiny (Turc et al., 2019)	yes	yes (8k)	4.4 M
Transformer, fine-tuned	BERT-small (Turc et al., 2019)	yes	yes (8k)	28.8 M
Transformer, fine-tuned	DistilBERT-base-uncased	yes	yes (8k)	67.0 M
Transformer, ready-made	DistilBERT fine-tuned on SST-2 (public)	yes	no	67.0 M
Transformer, ready-made	Twitter-RoBERTa sentiment (public)	yes	no	124.6 M
Transformer, zero-shot	NLI DistilRoBERTa cross-encoder	yes	no	82.1 M

The classical models represent the pre-neural approach and serve as strong, cheap baselines. The bidirectional LSTM represents the pre-Transformer neural approach, trained from scratch without pre-training. The three fine-tuned Transformers differ only in size (4.4, 29 and 67 million parameters), which allows us to observe how quality and cost scale with model size. BERT-tiny (2 layers, hidden size 128) and BERT-small (4 layers, hidden size 512) are compact BERT models pre-trained and released by Google together with the study of Turc et al. (2019); DistilBERT is the distilled version of BERT-base (Sanh et al., 2019). The three ready-made models represent the "use a public model as a service" scenario: a DistilBERT model that has already been fine-tuned on SST-2 by its authors (the best case of reuse), a RoBERTa model trained for sentiment on tweets (a different domain, Loureiro et al., 2022) and a zero-shot NLI model (Yin et al., 2019) that has never seen sentiment labels.

2.4. Evaluation criteria

Each model is evaluated by the following criteria:

– Accuracy, macro-averaged precision, recall and F1 score on the 872-sentence test set (quality).

– Training time in seconds on the same computer (cost of building the model).

– Inference latency in milliseconds per sentence, measured as total prediction time for the test set divided by the number of sentences, with batches of 32 (cost of using the model).

– Number of trainable parameters (memory footprint and storage).

– Accuracy on two linguistically difficult subsets: sentences containing negation words (not, no, never, n't, nothing, nor, neither) and sentences containing contrast markers (but, although, though, yet, however).

– Data efficiency: accuracy as a function of the number of labelled training examples (500, 1,000, 2,000, 4,000 and 8,000), measured for the TF-IDF logistic regression baseline and for the fine-tuned BERT-small model.

To check whether differences between models are statistically meaningful, 95% bootstrap confidence intervals of accuracy and McNemar's test for paired predictions are computed (Section 2.7.8).

All experiments were run on a laptop with an Intel Core i5-8365U processor (4 cores, 8 threads, 1.6 GHz base frequency), 16 GB of RAM and no GPU, under Windows 11, Python 3.11,

PyTorch 2 (CPU build), the Hugging Face Transformers library (Wolf et al., 2020) and scikit-learn. A fixed random seed (42) was used for data splits, sampling and model initialisation.

2.5. Design and structure of the developed program

The program is organised as a pipeline of independent Python scripts that communicate through files (Fig. 1). This design makes every stage reproducible separately and allows long-running training jobs to be executed in the background.

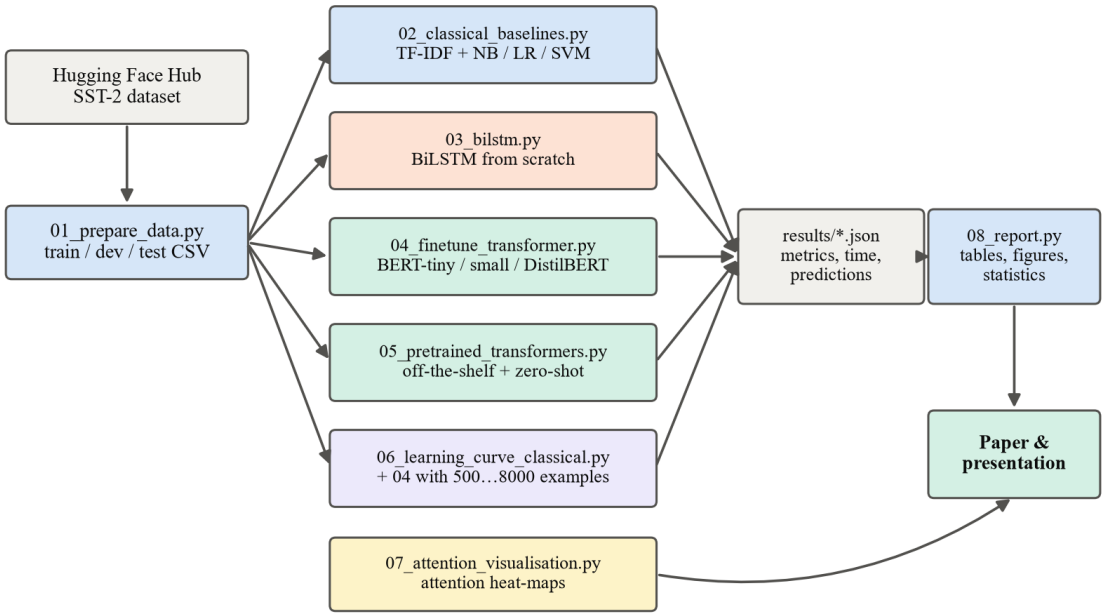


Fig. 1. Structure of the developed program (pipeline of Python modules)

The modules are:

– utils.py – shared functions: loading of data splits, computation of metrics, timing and saving of results in JSON format;

– 01_prepare_data.py – downloads SST-2 through the Hugging Face datasets library and writes the train, dev and test splits to CSV files;

– 02_classical_baselines.py – builds the TF-IDF representation and trains the naive Bayes, logistic regression and linear SVM classifiers;

– 03_bilstm.py – implements the tokeniser, the vocabulary and the BiLSTM network in PyTorch and trains it with early model selection on the dev set;

– 04_finetune_transformer.py – a generic fine-tuning program: given the name of any pre-trained encoder from the Hugging Face hub and the number of training examples, it adds a classification head and trains the whole network with AdamW and a linear warm-up schedule;

– 05_pretrained_transformers.py – evaluates the three ready-made models through the Transformers pipeline interface;

– 06_learning_curve_classical.py – trains the classical baseline on the same subsets as the Transformer learning curve;

– 07_attention_visualisation.py – extracts and plots the self-attention weights of a fine-tuned model for an example sentence;

– 08_report.py – collects all JSON results, produces the tables and figures, the subset (negation and contrast) analysis and the statistical tests.

Every model writes a result file with the same structure (model name, family, training size, training time, latency, number of parameters, metrics, confusion matrix and the list of predictions). Saving the individual predictions makes it possible to compute paired statistics and error analyses afterwards without re-running the models.

2.6. The Transformer architecture

This section describes the architecture used by all Transformer models in the experiment. The encoder of the original Transformer (Vaswani et al., 2017), which is the basis of BERT, DistilBERT and RoBERTa, is shown in Fig. 2. An encoder is a stack of L identical layers; each layer contains a multi-head self-attention sub-layer and a position-wise feed-forward sub-layer, both wrapped with a residual connection and layer normalisation.

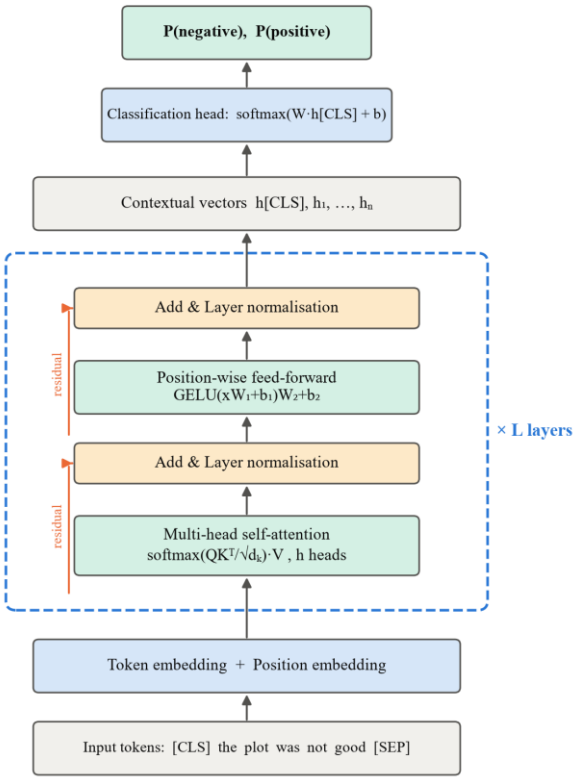


Fig. 2. Transformer encoder with a classification head, as used for sentiment classification

2.6.1. Tokenisation and input embeddings

Text is first split into sub-word units. BERT models use the WordPiece algorithm and RoBERTa uses byte-pair encoding. Frequent words are kept whole, while rare words are split into pieces (for example, "unwatchable" → "un", "##watch", "##able"), so the vocabulary is limited (about 30,000 units for BERT) and no word is ever out of vocabulary. A special [CLS] token is added at the beginning of every input and [SEP] at the end. Each token is mapped to a vector of dimension d (d = 128 for BERT-tiny, 512 for BERT-small and 768 for DistilBERT) by an embedding matrix E.

2.6.2. Positional encoding

Self-attention by itself is permutation-invariant: it treats a sentence as a bag of tokens. Information about word order is therefore added to the embeddings. The original Transformer used fixed sinusoidal encodings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (1)$$

where pos is the position of the token and i is the index of the dimension. BERT-type models instead learn a position embedding vector for each position up to 512. The input of the first layer is the sum of the token embedding and the position embedding: $h^{(0)}_t = E(w_t) + P(t)$.

2.6.3. Scaled dot-product attention

The central operation of the Transformer is attention. Every token representation is projected into three vectors: a query q, a key k and a value v. For a sequence of n tokens these are stacked into matrices Q, K, V $\in \mathbb{R}^{nxdk}$, obtained from the input H by learned projections:

$$Q = HW_Q, K = HW_K, V = HW_V \tag{2}$$

Attention is then computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{3}$$

The matrix product $Q \cdot K^T$ gives an n × n matrix of similarity scores between every pair of tokens. The softmax function, applied to each row, turns the scores into weights that are positive and sum to one:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_k e^{z_k}} \tag{4}$$

The output for each token is a weighted average of the value vectors of all tokens, so a word can directly gather information from any other word in the sentence regardless of distance. Division by $\sqrt{d_k}$ prevents the dot products from becoming very large for high dimensions, which would push the softmax into regions with extremely small gradients. In our task, for example, the representation of the word "good" in "not good" can attend strongly to "not", which allows the model to represent the negated meaning.

2.6.4. Multi-head attention

Instead of a single attention function, the Transformer runs h attention "heads" in parallel, each with its own projection matrices of dimension d_k = d / h:

$$\text{head}_i = \text{Attention}(HW_Q^i, HW_K^i, HW_V^i), \text{MultiHead}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \tag{5}$$

Different heads can specialise in different relations, for example one head may link adjectives with the nouns they modify while another links negation words with their scope (Clark et al., 2019). BERT-tiny uses 2 heads, BERT-small 8 heads and DistilBERT 12 heads per layer.

2.6.5. Feed-forward network, residual connections and layer normalisation

After attention, each position is processed independently by a two-layer feed-forward network:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \tag{6}$$

where the inner dimension is usually 4d and GELU is the Gaussian error linear unit activation used in BERT (the original Transformer used ReLU). Each sub-layer is wrapped by a residual connection followed by layer normalisation:

$$H' = \text{LayerNorm}(H + \text{MultiHead}(H)), H'' = \text{LayerNorm}(H' + \text{FFN}(H')) \tag{7}$$

$$\text{LayerNorm}(x) = \gamma \odot \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta \tag{8}$$

where μ and σ^2 are the mean and variance of the vector x, and γ and β are learned parameters. Residual connections allow gradients to flow through deep stacks, and normalisation stabilises training.

2.6.6. Computational complexity

Self-attention compares every token with every other token, so one layer requires $O(n^2 \cdot d)$ operations for a sequence of length n, compared with $O(n \cdot d^2)$ for a recurrent layer. However, all n positions are computed in parallel, whereas a recurrent network needs n sequential steps. For short and medium sequences ($n < d$), which is the case for sentences, self-attention is both faster and better at modelling long-range dependencies. For very long documents the quadratic cost becomes a limitation, which motivated efficient Transformer variants (Tay et al., 2022). In our experiment the maximum length was set to 64 tokens, which covers 99.99% of the training sentences and all test sentences (the longest test sentence has 55 sub-word tokens).

2.7. Mathematical methods of training and evaluation

2.7.1. Pre-training objective

BERT-type encoders are pre-trained by masked language modelling. A random 15% of the input tokens are selected; most of them are replaced by a [MASK] token, and the model is trained to predict the original tokens from the context. If M is the set of masked positions, the loss is

$$L_{MLM} = -\sum_{t \in M} \log P(w_t \mid x_{\setminus M}) \tag{9}$$

Because the model sees the context on both sides of the masked word, it learns deep bidirectional representations. DistilBERT is additionally trained to imitate the output distribution of the BERT-base teacher (knowledge distillation). Pre-training is extremely expensive (days on many GPUs or TPUs), therefore in this project we use publicly released pre-trained weights and only perform the fine-tuning step.

2.7.2. Fine-tuning for classification

For classification, the final hidden vector of the [CLS] token, $h_CLS \in \mathbb{R}^d$, is used as the representation of the whole sentence. A new classification layer with weights $W \in \mathbb{R}^{2xd}$ and bias b is added on top:

$$P(y \mid x) = \text{softmax}(W h_{CLS} + b) \tag{10}$$

All parameters (the pre-trained encoder and the new layer) are trained together by minimising the cross-entropy loss over the training set:

$$L_{CE} = -\frac{1}{N} \sum_{i=1}^{N} \log P(y_i \mid x_i) \tag{11}$$

2.7.3. Optimisation

Parameters are updated with AdamW (Loshchilov & Hutter, 2019), a variant of the Adam optimiser with decoupled weight decay. For gradient g_t at step t:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{12}$$

$$\theta_t = \theta_{t-1} - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \tag{13}$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected moment estimates, $\lambda = 0.01$ is the weight decay and η_t is the learning rate. The learning rate follows a linear schedule: it grows from 0 to its peak value during the first 10% of steps (warm-up) and then decreases linearly to 0. Warm-up prevents large, destructive updates of the pre-trained weights at the beginning of training. Gradients are clipped to a maximum norm of 1.0. The hyper-parameters used are shown in Table 5; they follow the recommendations of Devlin et al. (2019) and Sun et al. (2019), with a higher learning rate for the smallest models, which is usual for compact BERT variants.

Table 5. Fine-tuning hyper-parameters

Hyper-parameter	BERT-tiny	BERT-small	DistilBERT
Layers / hidden size / heads	2 / 128 / 2	4 / 512 / 8	6 / 768 / 12
Training examples	8 000	8 000	8 000
Epochs (best epoch kept by dev accuracy)	3	3	2
Peak learning rate	$2 \cdot 10^{-4}$	$1 \cdot 10^{-4}$	$5 \cdot 10^{-5}$
Batch size / maximum length	32 / 64	32 / 64	32 / 64
Warm-up / schedule	10% / linear	10% / linear	10% / linear
Weight decay / gradient clipping	0.01 / 1.0	0.01 / 1.0	0.01 / 1.0

2.7.4. TF-IDF representation

For the classical models every sentence is represented as a sparse vector of TF-IDF weights of unigrams and bigrams:

$$\text{tfidf}(t, d) = (1 + \log\text{tf}(t, d)) \cdot (\log\frac{1+N}{1+\text{df}(t)} + 1) \quad (14)$$

where tf(t, d) is the frequency of term t in sentence d (logarithmically damped), N is the number of sentences and df(t) is the number of sentences containing t. Each vector is normalised to unit length. Terms that appear in fewer than two training sentences are ignored. TF-IDF gives high weights to words that are frequent in a sentence but rare in the collection, but it ignores word order beyond bigrams and cannot represent the meaning of unseen words.

2.7.5. Classical classifiers

Multinomial naive Bayes applies Bayes' theorem with the assumption that features are conditionally independent given the class: $P(y \mid x) \propto P(y)\ \Pi_j\ P(t_j \mid y)^{(x_j)}$, with Laplace (additive) smoothing $\alpha = 0.5$.

Logistic regression models the probability of the positive class as $P(y = 1 \mid x) = \sigma(w^T x + b)$, where $\sigma(z) = 1 / (1 + e^{-z})$, and minimises the L2-regularised log-loss (inverse regularisation strength $C = 4$).

The linear support vector machine finds the hyperplane with the maximum margin by minimising the regularised hinge loss:

$$min_{w,b}\frac{1}{2}\|w\|^2 + C\ \sum_i max(0, 1 - y_i(w^T x_i + b)) \quad (15)$$

with labels coded as $y_i \in \{-1, +1\}$ and $C = 0.5$.

2.7.6. Bidirectional LSTM

The recurrent baseline processes the sequence of word embeddings with an LSTM cell (Hochreiter & Schmidhuber, 1997), defined by the gates:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (16)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, h_t = o_t \odot \tanh(c_t) \quad (17)$$

The forget, input and output gates (f_t, i_t, o_t) control what information is kept in the memory cell c_t. A bidirectional LSTM runs one LSTM from left to right and another from right to left and concatenates their hidden states. The sentence representation is obtained by max-pooling over time, followed by dropout ($p = 0.3$) and a linear output layer. Embeddings (dimension 128) and all weights are initialised randomly, i.e. the model has no pre-trained knowledge. It is trained for 4 epochs with Adam (learning rate 0.001, batch size 64), and the epoch with the best dev accuracy is kept.

2.7.7. Zero-shot classification by natural language inference

An NLI model receives a premise p and a hypothesis h and predicts whether p entails, contradicts or is neutral to h. For zero-shot classification (Yin et al., 2019), the sentence is used as the

premise and, for each candidate label c, a hypothesis is built from a template, here "The sentiment of this movie review is c." The entailment scores of the candidate labels are normalised with softmax, and the label with the highest entailment probability is predicted:

$$\hat{y} = \underset{c \in \{pos, neg\}}{\operatorname{argmax}} P(\text{entailment} \mid x, h(c)) \tag{18}$$

No sentiment-labelled data are used, so the method shows how much a Transformer can achieve purely from general language understanding.

2.7.8. Evaluation metrics and statistical tests

With TP, TN, FP and FN denoting true positives, true negatives, false positives and false negatives:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \text{Precision} = \frac{TP}{TP+FP}, \text{Recall} = \frac{TP}{TP+FN} \tag{19}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{20}$$

Precision, recall and F1 are computed for each class and averaged (macro average), so both classes have equal weight.

The test set contains only 872 sentences, so a difference of one percentage point corresponds to about nine sentences. To quantify uncertainty, 95% confidence intervals of accuracy are estimated with the bootstrap: the test set is resampled with replacement 2,000 times, accuracy is computed for each resample, and the 2.5th and 97.5th percentiles are taken. To test whether two models differ significantly, McNemar's test is applied to their paired predictions. If b is the number of sentences classified correctly only by model A and c the number classified correctly only by model B, then under the null hypothesis of equal accuracy b ~ Binomial(b + c, 0.5). We use the exact binomial version of the test, with significance level 0.05.

2.8. Experimental protocol and threats to validity

The experiment follows a fixed protocol so that all competitors are treated equally. (1) The data splits are created once and stored; every model reads the same CSV files. (2) Model selection uses only the dev set: the epoch with the highest dev accuracy is kept, and the test set is used exactly once per model for the final evaluation. (3) For the learning-curve experiment, the subsets of 500–8,000 training sentences are drawn with the same random seed for the classical and the Transformer model, so both learn from exactly the same examples. (4) Models are executed one after another on an otherwise idle computer, so that the measured training and inference times are not distorted by parallel jobs. (5) All hyper-parameters are fixed in advance from the literature and are not tuned on the test set.

Several threats to validity should be kept in mind when interpreting the results. Internal validity: each configuration was trained once with one random seed, therefore small differences (below about one percentage point) may be caused by random variation; the bootstrap intervals and McNemar tests are used to judge which differences are reliable. Construct validity: the negation and contrast subsets are defined by simple keyword rules, which capture most but not all such constructions. External validity: the results are obtained on English movie-review sentences; for other domains or for morphologically rich languages such as Lithuanian the absolute numbers will differ, although the relative behaviour of the model families is expected to be similar, as suggested by the Lithuanian studies reviewed in Chapter 1. Finally, the timing results depend on the hardware: on a GPU the training times of the Transformer models would be one to two orders of magnitude shorter, while the classical models would gain little.

3. Practical Part: Implementation and Results

3.1. Implementation of the program

The program described in Section 2.5 was implemented in Python 3.11 with PyTorch 2 (CPU build), Hugging Face Transformers and Datasets, scikit-learn and Matplotlib; it consists of about 770 lines of code in nine scripts and is published in the project repository (Appendix A). The data preparation script downloads SST-2 from the Hugging Face Hub and creates the three splits of Table 3. The classical baselines are scikit-learn pipelines (TF-IDF vectoriser followed by the classifier). The BiLSTM is a custom PyTorch module with its own tokeniser and vocabulary of the words occurring at least twice in the training data. The core of the work is the generic fine-tuning script: it loads any pre-trained encoder and its tokeniser by name, adds a two-class classification head, pads every batch dynamically to its longest sentence, and runs the training loop with AdamW, linear warm-up and decay, gradient clipping and selection of the best epoch on the dev set. The ready-made models are evaluated through the Transformers pipeline interface, and the zero-shot model through the zero-shot-classification pipeline with the hypothesis template of Section 2.7.7.

During development two practical problems had to be solved. The newest version of the Datasets library requires full repository names, so SST-2 is loaded as nyu-mll/glue. The compact BERT models of Turc et al. (2019) published by third parties on the Hub lacked tokenizer files compatible with the current library version, therefore the official Google checkpoints (google/bert_uncased_L-2_H-128_A-2 and L-4_H-512_A-8) were used. The whole pipeline is started by one command (run_all.sh) and executes all models one after another; every result is saved as a JSON file, from which the report script generates all tables and figures of this chapter.

3.2. Overall comparison of the models

The results of all ten models on the 872 test sentences are given in Table 6 and Fig. 3. Because the classes are almost balanced, accuracy and macro F1 differ by less than 0.2 percentage points (p.p.), so accuracy is used in the discussion.

Table 6. Results on the SST-2 test set (872 sentences), sorted by accuracy

Model	Train size	Acc., %	Prec., %	Rec., %	F1, %	Train time, s	Latency, ms
DistilBERT-SST2 (off-the-shelf)	0	91.1	91.1	91.0	91.0	–	44.6
DistilBERT fine-tuned (8k)	8 000	88.8	88.8	88.8	88.8	9530	39.1
Twitter-RoBERTa (off-the-shelf)	0	85.8	86.1	85.9	85.8	–	84.8
BERT-small fine-tuned (8k)	8 000	83.7	83.7	83.7	83.7	1581	14.1
TF-IDF + Linear SVM	65 349	82.9	83.1	82.8	82.9	2	0.06
BiLSTM (from scratch)	65 349	82.8	83.2	82.7	82.7	380	0.45
TF-IDF + Naive Bayes	65 349	82.2	82.8	82.1	82.1	2	0.05
TF-IDF + Logistic Regr.	65 349	81.9	82.2	81.8	81.8	4	0.05
Zero-shot NLI DistilRoBERTa	0	81.8	82.6	81.9	81.7	–	107.0
BERT-tiny fine-tuned (8k)	8 000	76.6	76.8	76.5	76.5	120	1.2

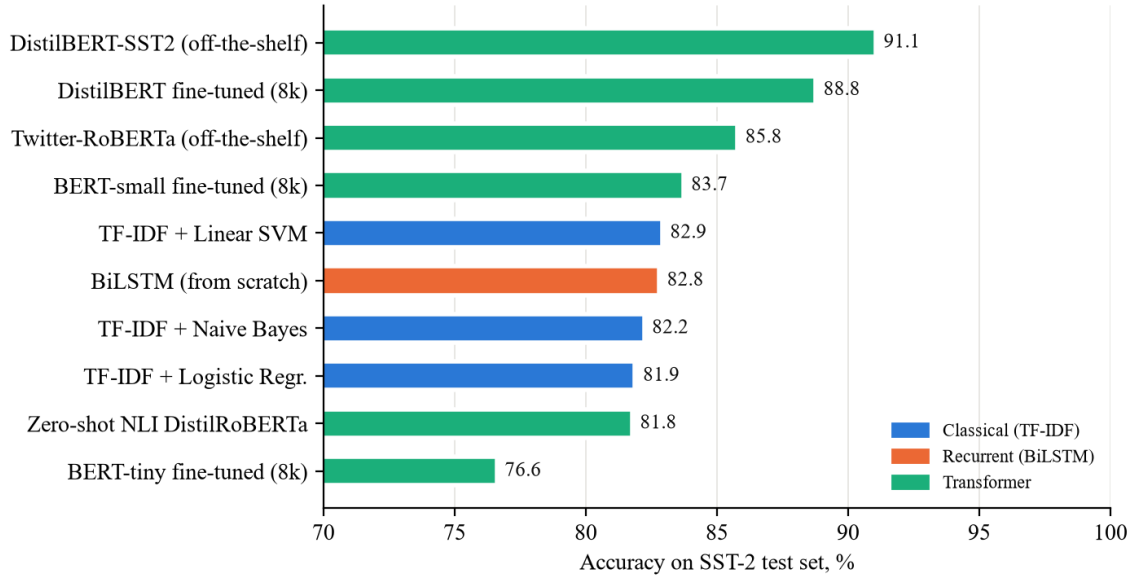


Fig. 3. Test accuracy of all compared models (colour shows the model family)

Three groups can be distinguished. The best results were obtained by the DistilBERT models: the publicly available model that its authors fine-tuned on the full SST-2 training set reached 91.1%, and the DistilBERT that we fine-tuned ourselves on only 8,000 sentences reached 88.8%. The out-of-domain Twitter-RoBERTa (85.8%) and our fine-tuned BERT-small (83.7%) form the middle group. The classical models, the BiLSTM and the zero-shot model are clustered in a narrow band between 81.8% and 82.9%. The smallest Transformer, BERT-tiny with only 4.4 million parameters, was the weakest model (76.6%), which shows that a Transformer is not automatically better: its capacity must be sufficient for the task.

Table 7 shows how reliable these differences are. With 872 test sentences the 95% bootstrap confidence intervals are about ±2.5 p.p. wide. All models are significantly worse than the best one (McNemar $p < 0.05$), including our fine-tuned DistilBERT ($p = 0.029$), which was trained on eight times less data. Pairwise tests against the best classical model (TF-IDF + linear SVM) show that our fine-tuned DistilBERT is significantly better: it correctly classifies 104 sentences that the SVM misses, while the opposite happens for only 53 sentences ($p < 0.001$). In contrast, the differences between BERT-small and the SVM (71 vs 64 sentences, $p = 0.61$), between the BiLSTM and the SVM ($p = 1.0$) and between the zero-shot model and logistic regression ($p = 1.0$) are not statistically significant. The answer to RQ1 is therefore: a sufficiently large pre-trained Transformer improves accuracy by about 6 p.p. over the best classical and recurrent models, even when trained on far fewer labelled examples, whereas compact Transformers only match the classical baselines.

Table 7. Bootstrap 95% confidence intervals and McNemar test against the best model

Model	Acc., %	95% CI	Only best correct	Only this correct	p-value
DistilBERT-SST2 (off-the-shelf)	91.1	[89.1; 92.9]	–	–	–
DistilBERT fine-tuned (8k)	88.8	[86.6; 90.7]	48	28	0.029
Twitter-RoBERTa (off-the-shelf)	85.8	[83.4; 88.1]	82	36	< 0.001
BERT-small fine-tuned (8k)	83.7	[81.1; 85.9]	90	26	< 0.001
TF-IDF + Linear SVM	82.9	[80.4; 85.3]	106	35	< 0.001
BiLSTM (from scratch)	82.8	[80.3; 85.2]	103	31	< 0.001
TF-IDF + Naive Bayes	82.2	[79.6; 84.7]	115	38	< 0.001
TF-IDF + Logistic Regr.	81.9	[79.2; 84.4]	114	34	< 0.001
Zero-shot NLI DistilRoBERTa	81.8	[79.1; 84.2]	116	35	< 0.001
BERT-tiny fine-tuned (8k)	76.6	[73.7; 79.2]	154	28	< 0.001

The BiLSTM trained from scratch reached 82.8%, practically identical to the TF-IDF models, although its training took 100–200 times longer. This agrees with the finding of Kapočiūtė-Dzikienė et al. (2019) for Lithuanian that neural networks without pre-training do not outperform well-tuned classical methods. The BiLSTM also showed clear overfitting to the training distribution: its dev accuracy (91.3%, measured on short phrases from the training distribution) was much higher than its test accuracy on complete sentences.

3.3. Cost: training time, inference latency and model size

Fig. 4 relates accuracy to inference latency (RQ2). The classical models classify a sentence in about 0.05 ms and train in 2–4 seconds on all 65 thousand sentences. The BiLSTM needs 0.45 ms per sentence and 6 minutes of training. The fine-tuned Transformers are slower by two to three orders of magnitude: BERT-tiny needs 1.2 ms, BERT-small 14 ms and DistilBERT 39 ms per sentence, and the larger ready-made models 45–107 ms. The zero-shot model is the slowest because it must process every sentence once for each candidate label.

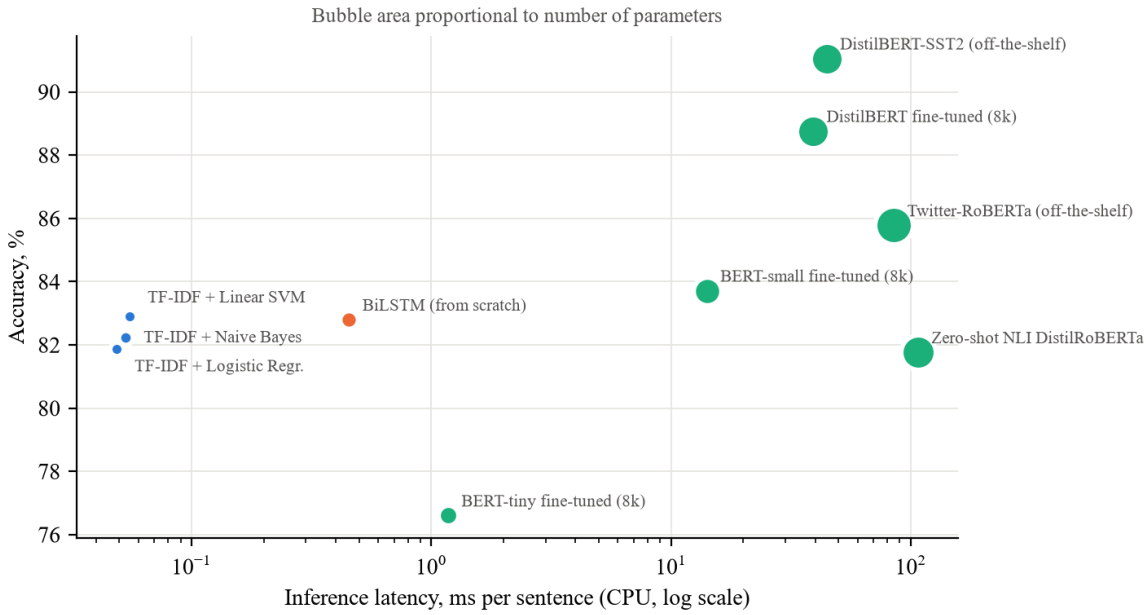


Fig. 4. Accuracy versus inference latency on CPU (bubble area proportional to the number of parameters)

Fine-tuning on the laptop CPU took 2 minutes for BERT-tiny, 26 minutes for BERT-small and about 2.6 hours for DistilBERT (8,000 sentences, 2–3 epochs). These times should be read as upper bounds: the laptop was also used for other tasks while the longest runs were executing, which slowed them down. Even so, the conclusion is clear: on a CPU, a compact or distilled Transformer can be fine-tuned within hours, and its inference speed (about 25 sentences per second for DistilBERT) is sufficient for most interactive applications, but not for very high-volume streams, where TF-IDF models remain hundreds of times cheaper. In terms of size, the TF-IDF models have about 74 thousand weights (one per n-gram), the BiLSTM 2 million parameters, and the Transformers 4.4–125 million parameters (17–500 MB of memory in 32-bit precision).

3.4. Data efficiency of pre-training

The learning-curve experiment (RQ3) is summarised in Table 8 and Fig. 5. Both models were trained on exactly the same random subsets.

Table 8. Test accuracy as a function of the number of labelled training examples

Training examples	BERT-small fine-tuned, %	TF-IDF + Logistic Regr., %	Difference, p.p.
500	75.9	61.8	+14.1
1 000	77.6	68.1	+9.5
2 000	80.7	71.0	+9.7
4 000	82.7	75.8	+6.9
8 000	83.7	77.5	+6.2

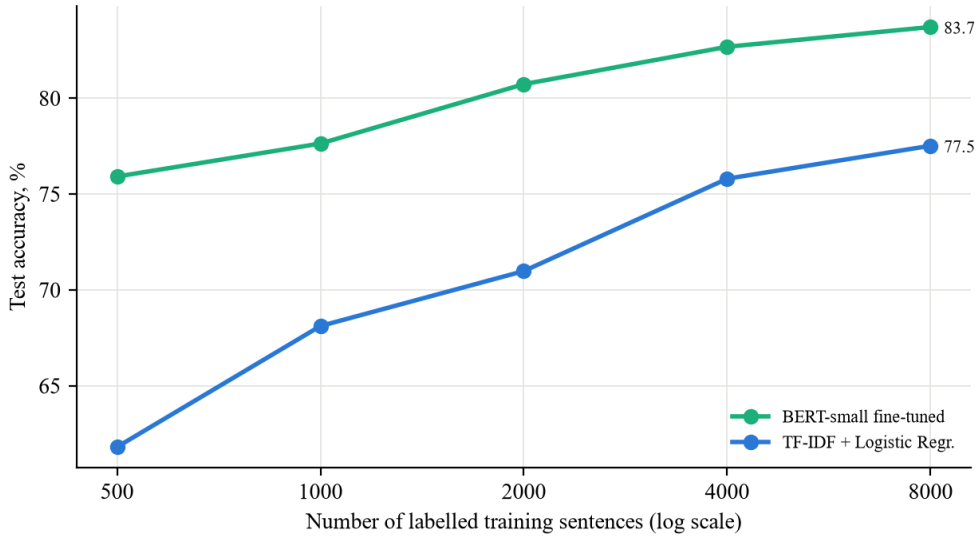


Fig. 5. Learning curves: test accuracy versus number of labelled training sentences

The advantage of pre-training is largest when labelled data are scarce. With only 500 labelled sentences BERT-small reaches 75.9%, while the TF-IDF model reaches only 61.8%, a difference of 14.1 p.p.; the TF-IDF model simply has not seen most of the words of the test sentences, whereas the pre-trained model already knows them. The difference decreases as the data grow (6.9 p.p. at 4,000 and 6.2 p.p. at 8,000 sentences). BERT-small trained on 4,000 sentences (82.7%) already matches the best classical model trained on all 65,349 sentences (82.9%), i.e. it achieves the same quality with about 16 times less labelled data, and DistilBERT trained on 8,000 sentences exceeds it by 5.9 p.p. This result is directly relevant for less-resourced languages such as Lithuanian, where large labelled datasets rarely exist.

3.5. Ready-made and zero-shot models

The ready-made models answer RQ4. When a public model has been fine-tuned on the same domain and task, it is the best option: DistilBERT-SST2 achieved the highest accuracy without any training on our side. A model trained for the same task in a different domain (Twitter-RoBERTa) lost 5.3 p.p. compared with it, although it still beat all classical models; the informal style of tweets differs from the language of film critics. The zero-shot NLI model, which has never seen a sentiment label, reached 81.8% – the same level as TF-IDF logistic regression trained on 65 thousand labelled sentences. This illustrates the general language understanding acquired during pre-training: the task can be specified by a natural-language hypothesis instead of training data, at the price of the highest inference cost.

3.6. Error analysis: negation and contrast

To answer RQ5, accuracy was measured on 184 test sentences containing negation and 130 sentences containing contrast markers (Table 9, Fig. 6).

Table 9. Accuracy (%) on linguistically difficult subsets of the test set

Model	All	Negation	Contrast	Δ negation, p.p.
DistilBERT-SST2 (off-the-shelf)	91.1	87.0	86.2	-4.1
DistilBERT fine-tuned (8k)	88.8	85.3	85.4	-3.4
Twitter-RoBERTa (off-the-shelf)	85.8	83.7	84.6	-2.1
BERT-small fine-tuned (8k)	83.7	74.5	76.2	-9.3
TF-IDF + Linear SVM	82.9	74.5	77.7	-8.5
BiLSTM (from scratch)	82.8	78.8	77.7	-4.0
TF-IDF + Naive Bayes	82.2	73.9	76.9	-8.3
TF-IDF + Logistic Regr.	81.9	73.9	76.9	-8.0
Zero-shot NLI DistilRoBERTa	81.8	81.5	79.2	-0.2
BERT-tiny fine-tuned (8k)	76.6	69.6	65.4	-7.0

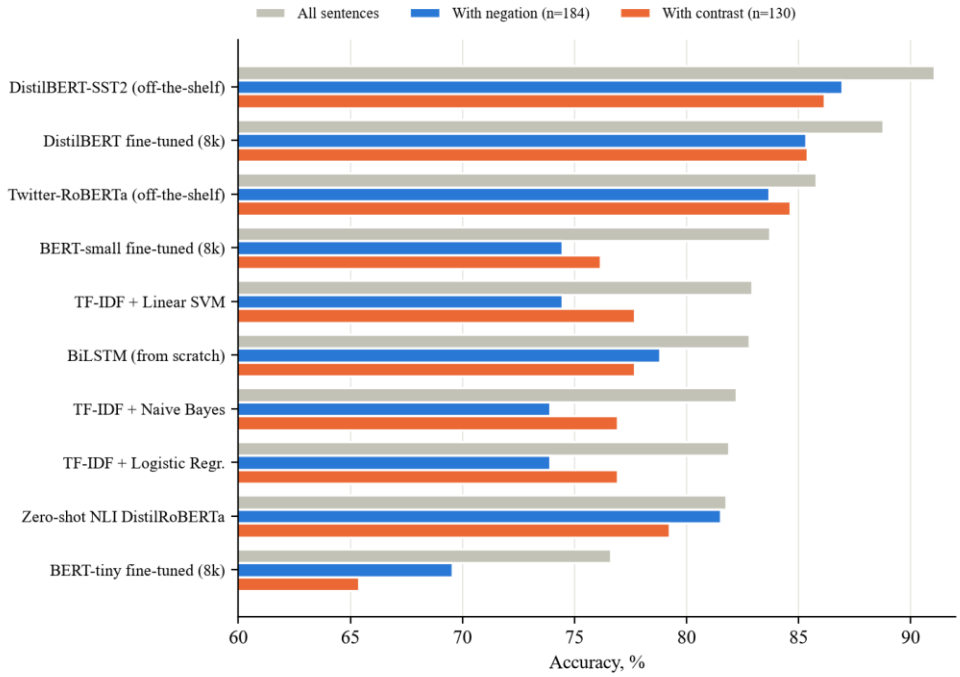


Fig. 6. Accuracy on all sentences and on sentences with negation and contrast

The bag-of-words models lose 8–9 p.p. on sentences with negation, because a sentence such as "not good" contains the positive word "good". The large Transformers are much more robust: DistilBERT-SST2 loses 4.1 p.p., our fine-tuned DistilBERT 3.4 p.p. and Twitter-RoBERTa 2.1 p.p. The zero-shot NLI model is the most robust of all (−0.2 p.p.), which is plausible because NLI training specifically teaches a model to handle negation and contradiction. The compact Transformers, however, behave like the classical models: BERT-small loses 9.3 p.p. on negation, and BERT-tiny 7.0 p.p. Interestingly, the BiLSTM, which reads the sentence in order, handles negation better (−4.0 p.p.) than TF-IDF. The advantage of Transformers on difficult sentences is therefore not a property of the architecture alone but of the combination of architecture, model capacity and pre-training.

Fig. 7 shows the self-attention weights of DistilBERT-SST2 for the sentence "the plot was not good, but the acting was brilliant". The model classifies the sentence as positive with a probability of 1.000, i.e. it gives the clause after "but" the decisive weight, which agrees with how people read such sentences. In the last layer much of the attention of all words goes to the comma that separates the two clauses and to the [SEP] token; such "attention sinks" on punctuation and separator tokens were also observed in BERT by Clark et al. (2019). The words of the second clause ("acting", "was", "brilliant") spread their remaining attention mainly over the other words of the same clause. As noted

by Jain and Wallace (2019), such pictures illustrate the information flow but should not be treated as a complete explanation of the decision.

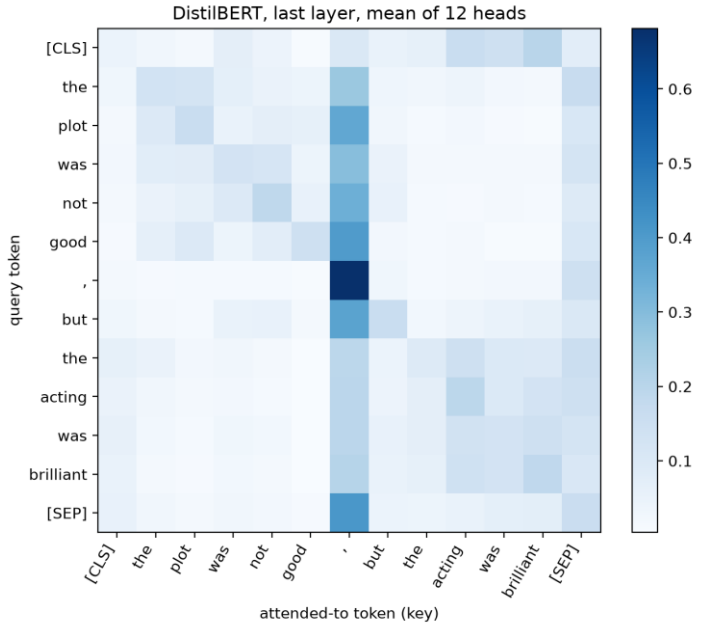


Fig. 7. Self-attention weights of DistilBERT for an example sentence (last layer, mean over heads)

Examples from the test set show the typical behaviour. The fine-tuned DistilBERT correctly classified 104 sentences that the SVM misclassified, for example "in its best moments, resembles a bad high school production of grease, without benefit of song" (negative; the SVM is misled by "best") and "the subtle strength of elling is that it never loses touch with the reality of the grim situation" (positive; "grim" and "never loses" mislead the bag-of-words model). The SVM was right in 53 cases where DistilBERT failed, for example in sentences with mixed sentiment such as "we root for (clara and paul), even like them, though perhaps it's an emotion closer to pity".

3.7. Comparison with previous research

Our results are consistent with the literature summarised in Table 2. The classical models (81.9–82.9%) match the 79–83% reported for bag-of-words models on SST-2 (Socher et al., 2013), and the off-the-shelf DistilBERT (91.1%) is close to the 91.3% reported by Sanh et al. (2019) on the same validation set, which confirms that our evaluation procedure is correct. Our fine-tuned DistilBERT stays 2.3 p.p. below it because it was trained on 12% of the training data. Compared with the BERT-base result of 93.5% (Devlin et al., 2019), our models are smaller and trained with fewer data, which explains the gap. The new element of this study is the joint view of quality, cost and data efficiency on a CPU-only computer, which shows when the advantage of Transformers is worth its price.

Conclusions

1. The literature review showed that Transformer models are the state of the art for text classification: fine-tuned encoders exceed 90% accuracy on SST-2, compared with 79–83% for classical models. Lithuanian research confirms that neural networks without pre-training do not necessarily outperform classical methods, while pre-trained models (LitLat BERT, multilingual BERT and T5, Lithuanian Llama2) are now becoming available. The gap identified in the literature was the lack of studies that jointly evaluate quality, cost and data efficiency of Transformers under realistic resource limits.

2. The analysis of the architecture showed that the key advantages of the Transformer are self-attention, which connects every pair of words directly, parallel processing of the whole sequence, and the possibility of large-scale self-supervised pre-training followed by inexpensive fine-tuning. Its main limitations are the quadratic cost of attention with respect to sequence length and the large number of parameters.

3. A reproducible methodology was defined and implemented: ten models from four families (classical TF-IDF, recurrent BiLSTM, fine-tuned Transformers and ready-made Transformers) were compared on the same SST-2 splits on a CPU-only laptop, using accuracy, macro F1, training time, latency, model size, linguistic subsets, learning curves, bootstrap confidence intervals and McNemar tests.

4. The experiment showed that a sufficiently large pre-trained Transformer is clearly the most accurate approach. DistilBERT fine-tuned on only 8,000 sentences reached 88.8% accuracy, significantly more than the best classical model trained on all 65,349 sentences (TF-IDF + linear SVM, 82.9%, $p < 0.001$), and the ready-made in-domain DistilBERT reached 91.1%. Compact Transformers only matched the classical models (BERT-small 83.7%), and BERT-tiny was the weakest model (76.6%). The BiLSTM trained from scratch (82.8%) gave no advantage over the classical models.

5. The improvement has a clear cost: Transformers are 20–2,000 times slower at inference (1.2–107 ms vs 0.05 ms per sentence on a CPU) and 60–1,700 times larger than TF-IDF models, and fine-tuning on a CPU takes from minutes to hours. In return, pre-training makes them far more data-efficient – with 500 labelled sentences BERT-small was 14.1 percentage points more accurate than TF-IDF, and with 4,000 sentences it matched the classical model trained on 16 times more data – and the larger Transformers were much more robust on sentences with negation and contrast (accuracy drop of 2–4 p.p. versus 8–9 p.p. for bag-of-words models).

Practical recommendations. (1) First check whether a public model already fine-tuned for the same task and domain exists – it gives the best quality without any training. (2) If not, fine-tune a distilled Transformer such as DistilBERT on a few thousand labelled examples; this is feasible even without a GPU. (3) When no labelled data exist at all, a zero-shot NLI model gives a usable baseline comparable to classical models. (4) TF-IDF models remain the right choice only when inference must be extremely cheap or the hardware is very limited. For Lithuanian texts, the same procedure can be applied with LitLat BERT or XLM-RoBERTa, which is the most natural continuation of this work, together with repeated runs with several random seeds and GPU-based timing.

References

1. Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015). arXiv:1409.0473.
2. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.
3. Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). What does BERT look at? An analysis of BERT's attention. In Proceedings of the 2019 ACL Workshop BlackboxNLP (pp. 276–286).
4. Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). ELECTRA: Pre-training text encoders as discriminators rather than generators. In International Conference on Learning Representations (ICLR 2020).
5. Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., et al. (2020). Unsupervised cross-lingual representation learning at scale. In Proceedings of the 58th Annual Meeting of the ACL (pp. 8440–8451).
6. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT 2019 (pp. 4171–4186).
7. He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with disentangled attention. In International Conference on Learning Representations (ICLR 2021).
8. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
9. Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. In Proceedings of the 56th Annual Meeting of the ACL (pp. 328–339).
10. Jain, S., & Wallace, B. C. (2019). Attention is not explanation. In Proceedings of NAACL-HLT 2019 (pp. 3543–3556).
11. Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. In Machine Learning: ECML-98, LNCS 1398 (pp. 137–142). Springer.
12. Kapočiūtė-Dzikienė, J., Damaševičius, R., & Woźniak, M. (2019). Sentiment analysis of Lithuanian texts using traditional and deep learning approaches. Computers, 8(1), 4. https://doi.org/10.3390/computers8010004
13. Kapočiūtė-Dzikienė, J., Krupavičius, A., & Krilavičius, T. (2013). A comparison of approaches for sentiment classification on Lithuanian Internet comments. In Proceedings of the 4th Biennial International Workshop on Balto-Slavic Natural Language Processing (pp. 2–11). ACL.
14. Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2020). ALBERT: A lite BERT for self-supervised learning of language representations. In International Conference on Learning Representations (ICLR 2020).
15. Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In Proceedings of the 58th Annual Meeting of the ACL (pp. 7871–7880).
16. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692.
17. Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In International Conference on Learning Representations (ICLR 2019).
18. Loureiro, D., Barbieri, F., Neves, L., Espinosa Anke, L., & Camacho-Collados, J. (2022). TimeLMs: Diachronic language models from Twitter. In Proceedings of the 60th Annual Meeting of the ACL: System Demonstrations (pp. 251–260).
19. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
20. Nakvosas, A., Daniušis, P., & Mulevičius, V. (2024). Open Llama2 model for the Lithuanian language. arXiv preprint arXiv:2408.12963.
21. Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. In Proceedings of NAACL-HLT 2018 (pp. 2227–2237).
22. Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. OpenAI technical report.

23. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research, 21(140), 1–67.
24. Rogers, A., Kovaleva, O., & Rumshisky, A. (2020). A primer in BERTology: What we know about how BERT works. Transactions of the Association for Computational Linguistics, 8, 842–866.
25. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.
26. Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A. Y., & Potts, C. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. In Proceedings of EMNLP 2013 (pp. 1631–1642).
27. Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In Proceedings of the 57th Annual Meeting of the ACL (pp. 3645–3650).
28. Sun, C., Qiu, X., Xu, Y., & Huang, X. (2019). How to fine-tune BERT for text classification? In Chinese Computational Linguistics (CCL 2019), LNCS 11856 (pp. 194–206). Springer.
29. Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2022). Efficient transformers: A survey. ACM Computing Surveys, 55(6), 1–28.
30. Turc, I., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Well-read students learn better: On the importance of pre-training compact models. arXiv preprint arXiv:1908.08962.
31. Ulčar, M., & Robnik-Šikonja, M. (2021). Training dataset and dictionary sizes matter in BERT models: The case of Baltic languages. arXiv preprint arXiv:2112.10553.
32. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.
33. Vileikytė, B., Lukoševičius, M., & Stankevičius, L. (2024). Sentiment analysis of Lithuanian online reviews using large language models. In Proceedings of IVUS 2024, CEUR Workshop Proceedings, 3885, 249–258.
34. Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2019). GLUE: A multi-task benchmark and analysis platform for natural language understanding. In International Conference on Learning Representations (ICLR 2019).
35. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. In Proceedings of EMNLP 2020: System Demonstrations (pp. 38–45).
36. Yin, W., Hay, J., & Roth, D. (2019). Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In Proceedings of EMNLP-IJCNLP 2019 (pp. 3914–3923).
37. Zhang, L., Wang, S., & Liu, B. (2018). Deep learning for sentiment analysis: A survey. WIREs Data Mining and Knowledge Discovery, 8(4), e1253.
38. Zhang, W., Deng, Y., Liu, B., Pan, S. J., & Bing, L. (2024). Sentiment analysis in the era of large language models: A reality check. In Findings of the Association for Computational Linguistics: NAACL 2024.

Appendices

Appendix A. Source code and reproduction of the results

The complete source code, the results of every model (JSON files with metrics, timings and predictions), the figures, this paper and the presentation are available in the public repository https://github.com/shashikanth077/Transformers-NLP. The repository is organised as follows:

– code/ – the experiment: 01_prepare_data.py … 09_diagrams.py, utils.py, run_all.sh, requirements.txt;

– code/results/ – one JSON file per model and the summary tables (summary_table.csv, statistics.csv, subset_accuracy.csv, learning_curve.csv);

– code/figures/ – all figures used in the paper and the presentation;

– paper/ – the chapter texts and build_docx.py, which generates this document from the VILNIUS TECH template;

– slides/ – build_deck.js, which generates the presentation from the results;

– docs/ – the final paper (DOCX, PDF) and the presentation (PPTX).

To reproduce the experiment, create a Python 3.11 environment, install the CPU build of PyTorch and the packages listed in code/requirements.txt, and run bash run_all.sh in the code directory. On a 4-core laptop the whole pipeline takes about 2.5–3 hours; the dataset and the pre-trained models (about 1.5 GB) are downloaded automatically from the Hugging Face Hub on the first run.